3.1 Technologies Used:

The Empower-Her system was implemented with the following programming language, frameworks, libraries, cryptographic techniques and tools.

Python

Python 3.12 is the programming language of the backend. It is used for request handling, data processing, recommendation generation, document processing and the cryptographic algorithms. The project supports Python 3.11 to 3.13.

Django and Django REST Framework

Django 5.2 is the web framework. It manages user registration, authentication, access control, database interactions and the web pages. Django REST Framework provides the REST APIs under /api/ through which every page loads and submits its data. Logins use tokens that expire after seven days and are rate limited. The code is organised into eleven Django apps, one per feature, kept separate from the frontend pages.

Artificial Intelligence and Machine Learning

Government schemes are recommended by combining eligibility rules (state, business stage, social group and amount) with TF-IDF text similarity from scikit-learn between the user's business description and each scheme. Business idea validation combines a financial model, rule-based analysis and census market indicators. When an API key is configured, an OpenAI Large Language Model adds written recommendations and conversational answers, using only the facts supplied by Empower-Her; without a key, the system answers from its own data.

Privacy and Security Technologies

- **Homomorphic Encryption:** the Paillier cryptosystem with 2048-bit keys, implemented in Python on the server and in JavaScript in the browser. The server computes loan affordability on encrypted income and expense figures.
- **Zero-Knowledge Proofs:** Pedersen commitments with an Okamoto (Schnorr-style) proof of knowledge, made non-interactive with the Fiat-Shamir method, in the 2048-bit group of RFC 3526. They prove ownership of a registered idea without revealing it.
- **Encryption at rest:** Fernet (AES-128 with HMAC-SHA256) encrypts confidential ideas, uploaded documents and their OCR text, using separate keys.

- **Application security:** PBKDF2 password hashing, expiring tokens, rate limiting, owner-only data access, and HTTPS, HSTS and secure cookies in production.

Optical Character Recognition (OCR)

RapidOCR, which runs PaddleOCR models on ONNX Runtime, extracts text from document images, and pypdf extracts text from PDF files. The extracted numbers are validated: the PAN format, the Aadhaar Verhoeff checksum, the GSTIN check character, and the Udyam and IFSC formats. Only masked numbers are stored.

Recommendation System and Market Analysis

Pandas and NumPy process the datasets: Census 2011 indicators for 640 districts, Ministry of MSME data on women-owned enterprises by state, and a curated dataset of 25 central and state government schemes with their official sources. The market module converts census indicators into percentiles and weights them by business category to give a demand potential index.

Voice Assistance

The assistant answers in English, Telugu and Hindi. Speech recognition and speech synthesis use the Web Speech API built into browsers such as Chrome and Edge, so no additional speech software is installed on the server.

Database Management

SQLite stores user accounts, profiles, business assessments, Idea Vault entries and encrypted documents through the Django ORM. PostgreSQL can be used for larger deployments.

Frontend Development

The user interface is built with Django templates, HTML, CSS and plain JavaScript, with a responsive layout for mobile phones. This replaced the React and Bootstrap frontend considered in the proposal, so that a single Django server delivers both the pages and the APIs, which simplifies setup and deployment.

Deployment and Testing

One-command setup scripts prepare the environment, encryption keys, database and datasets on Windows, Linux and macOS. In production the application runs on the gunicorn server with WhiteNoise for static files. Each module has automated tests written with the Django test framework.

3.2 Development Process

Empower-Her was developed with a modular approach. Each feature was built as a separate Django app with its own API, logic, data and tests, and then integrated through the dashboard. The six stages of the methodology in Figure 3.1 were implemented as follows:

- **User input and registration:** account registration, login and an entrepreneur profile (state, city, business stage and preferred language) that personalises the other modules.
- **Data processing and security:** input validation, owner-only access, encryption of confidential data, homomorphic encryption for financial figures, and zero-knowledge proofs for ideas.
- **AI processing and analysis:** business idea validation, scheme recommendation, local market demand analysis and optional LLM recommendations.
- **Document processing and verification:** OCR of identity and business documents with number validation and encrypted storage.
- **Additional entrepreneurial services:** EMI and affordability calculators, the PMEGP subsidy calculator, the multilingual assistant and women safety resources.
- **Personalised dashboard:** web pages for every module, reachable from a single dashboard and usable on mobile phones.

A knowledge base of government schemes, loan products, census and MSME data, and safety resources supports the recommendations and analysis. Each module was tested independently with automated tests and checked in the browser before integration. The system was then hardened for security and production use. The outcomes of this process are presented in Chapter 4.